

SLAM 3.0 API

- 串口通讯

Plain Text

```
1 串口波特率：115200
2
3 导航主机串口：/dev/XX
4 **指令包：**
5 - 通信方式为串行异步方式，ROS系统和上位机的通讯内容为ASCII编码字符串。
6
7 | 帧头(H) | 长度(L) | 数据(D) | 校验位(S) |
8 | :-- | :-----: | :-----: | :-----: | :-----: |
9 | AA 54 | | | | |
10
11
12 **指令包数据定义：**
13 - 帧头(H)： AA 54
14 - 长度(L)： 数据(D) 的长度
15 - 数据(D)： 字符串转换的byte数组
16 - 校验位(S)：长度(L)、数据(D) 的异或计算
17
18
19 **应答包：**
20 - 通信方式为串行异步方式，ROS系统和上位机的通讯内容为ASCII编码字符串。
21
22 | 帧头(H) | 长度(L) | 数据(D) | 校验位(S) |
23 | :-- | :-----: | :-----: | :-----: | :-----: |
24 | AA 54 | | | | |
25
26
27 **应答包数据定义：**
28 - 帧头(H)： AA 54
29 - 长度(L)： 数据(D) 的长度
30 - 数据(D)： 字符串转换的byte数组
31 - 校验位(S)：长度(L)、数据(D) 的异或计算
```

• ROS 主动上报

1. imu, 激光, 里程计, 3d 摄像头状态

app 在收到这几个状态异常时应该做出相应提示,如果处于导航状态,应停止导航,结束任务;

Plain Text

```
1 response: check_sensors{imu lidar odom 3dcam ros lxcam}
2 mark:
3     imu: imu状态;
4     lidar: 激光雷达状态;
5     odom: 里程计状态;
6     3dcam: 3d摄像头状态;
7     ros: ros服务状态;
8     lxcam: 卡板识别相机状态(叉车);
9     0:异常;1:正常;
10    导航系统主动上报,每5s一次
```

2. 导航状态

导航结果

Plain Text

```
1  response:
2      nav_result{state code name dist_to_goal mileage}
3  mark:
4      state:
5          0:初始状态;
6          1:机器开始导航;
7          2:暂停;
8          code:
9              0 : 成功;
10             -1 : 失败;
11      3:导航结果;
12          code:
13              0 : 成功;
14              -1 : 失败;
15      4:导航取消;
16          code:
17              0 : 成功;
18      5:导航恢复;
19          code:
20              0 : 成功;
21              -1 : 失败;
22      6:上位机发送导航指令成功,但是还没有开始导航;
23          code:
24              0 : 成功;
25              -1 : 正在对接充电桩;
26              -2 : 急停开关被按下;
27              -3 : 适配器充电中;
28              -4 : 找不到目标点;
29              -5 : agv 对接中;
30              -6 : 定位异常;
31              -7 : 固定路线点位间距过大;
32              -8 : 找不到固定路线;
33              -9 : 读取点位信息失败;
34              -10 : 未退出对接状态;
35      name:
36          目标点导航 : 目标点名称;
37          坐标导航 :
38              x:x轴坐标;
```

```
39         y : y轴坐标;
40         radian : 弧度
41     dist_to_goal : 距离目标点的距离;
42     mileage : 导航里程;
43
44     导航流程示例:
45         send : nav_point[A]//发送导航目标点
46         resp : nav_result{6 0 A -1 0}//开始导航
47         resp : nav_result{1 0 A 0.562001 0}
48         resp : nav_result{3 0 A 0 0}//导航结果
49         resp : nav_result{0 0 -1 -1 0}
50
```

3. 导航错误状态

Plain Text

```
1 response: move_status:x
2 mark:
3     x:
4         3: 规划不了路线;
5         4: 局部路径有障碍;
6         5: 单次导航失败, 自动重新开始导航 (固定路线)
```

4. 核心状态

Plain Text

```
1 response: core_data{data1 data2 data3 data4 data5}
2 mark:
3     data1: 碰撞:
4         1: center; 2: back; 4: left; 8: right;
5     data2: 防跌:
6         4: left; 2: 急刹; 1: 3D;
7     data3: 急停:
8         0: 按下松轴; 1: 抬起导航;
9     data4: 电量:
10        电量百分比;
11     data5: 充电状态:
12        1: 未充电; 2: 座充; 3: 适配器; 8: 对接中; 9: 对接时未找到充电桩; 11: 超
        时没有收到红外码; 12: 对接充电桩第二阶段没有收到红外码; 13: 对接充电桩第三阶段没有收
        到红外码
```

5. 轮子状态

Plain Text

```
1 response:
2   wheel_status{work_state current_left current_right temp_left temp_r
   ight driver_temp_left driver_temp_right code_left code_right model vers
   ion}
3 mark:
4   work_state : 工作状态
5       松轴 0x00
6       锁轴0x01
7       轮子故障 0x02
8       刹车0x04
9       停止(stop) 0x08
10      离线状态 0x80
11      current_left : 左轮电流 (mA);
12      current_right : 右轮电流 (mA);
13      temp_left : 左轮温度 (°C);
14      temp_right : 右轮温度 (°C);
15      driver_temp_left : 左轮驱动器温度 (°C);
16      driver_temp_right : 右轮驱动器温度 (°C);
17      code_left : 左轮错误码;
18      code_right : 右轮错误码;
19      model : 轮子型号;
20      version : 轮子版本号;
```

6. 点位更新

Plain Text

```
1 response: waypoint:update
2 mark: 点位更新时上报
```

7. 固定路线更新

Plain Text

```
1 response: pathmodel:update
2 mark: 固定路线更新时上报(3.2.2前不支持)
```

8. 关机

Plain Text

```
1 response: power_off:x
2 mark:
3     x : 1 : 软件关机; 2 : 重启; 3 : 按键关机; 4 : 低电关机;
4     收到关机上报需安卓关机;
```

9. 导航中上报速度

Plain Text

```
1 response: base_vel[line_speed angular_speed]
2 mark:
3     line_speed : 线速度;
4     angular_speed : 角速度;
```

10. 对接充电桩时上报后超声波

Plain Text

```
1 response:
2 {data1 data2 data3}
3 mark:
4     data1 : 后超声波;
```

11. 全局路径

Plain Text

```
1 response: global_path: & global_path1:
2 mark:
3     global_path:全局路径,一次如果不能完全上报会分两次上报(global_path会以+结尾)
```

12.agv 坐标上报

Plain Text

```
1 response: agv_tag_pose{tag_name pose1 pose2 pose3 poseX,poseY,poseAngle}
2     mark:
3         tag_name: 标签码名称;
4         poseX: X轴偏移量;
5         poseY: Y轴偏移量;
6         poseAngle: 角度;
```

13.agv 导航结果

进行agv导航时,ros会先上报nav_result,表示导航到目标点成功,接下来会根据二维码调整位置,然后进行二维码导航,此处为二维码导航的结果;

Plain Text

```
1 agv导航成功:
2     response: agv_success{tag}
3     mark:
4         agv导航成功;
5         tag: 目标标签码;
6 agv导航失败:
7     response: agv_fail
8     mark:
9         agv导航失败;
```

14.定位异常

Plain Text

```
1 response: misspose:code
2 mark:
3     code:
4         0:重新定位，定位重置
5         1:丢定位
```

15.生成固定路线

Plain Text

```
1 request: short_dij:& short_dij1:
2 mark:
3     一次如果不能完全上报会分多次上报(未上报完成时会以+结尾)
```

16.上报全局路径穿过的特殊区

Plain Text

```
1 response: special_plan:{"sp":[{"n":"sp-b","c":[-6.03,-0,-8.66,-2.08],"t
  ype":0}]}
2 mark:
3     n : 特殊区名称;
4     c : 全局路线和特殊区的交点坐标;
5     type : 特殊区类型:
6         0 : 默认;
7         1 : 单行区;
8         2 : 门控区;
9         3 : 转弯区;
10        4 : 自定义;
```

17.进入特殊区

Plain Text

```
1 response: special_area[sp-a,0,-1.0]
2 mark :
3     sp-a : 特殊区名称;
4     0 : 特殊区类型;
5     -1.0 : 经过特殊区时的速度,-1为默认速度;
```

18.离开特殊区

Plain Text

```
1 response: special_area:out
```

洁霸独有

1. 清洁进度

Plain Text

```
1 response: clean_room[running:progress,cleanroom_id]
2 mark:
3     progress:进度;
```

2. 清洁完成

Plain Text

```
1 response: clean_room[complete:code,clean_id]
2 mark:
3     code:
4         0:完成;
```

叉车独有

1. 叉车相关状态

JSON

```
1 response: forklift{arm_location position_sensor dock_state control_state reserved}
2     arm_location: 叉车臂位置
3         0: 中间;
4         1: 顶部;
5         2: 底部;
6         3: 异常;
7     position_sensor: 卡板到位传感器
8         0: 未到位;
9         1: 到位;
10    dock_state: 对接状态
11        0: 初始状态;
12        1: 对接中;
13        2: 对接成功;
14        3: 横向偏差调整成功;
15        4: 视觉识别不到托盘;
16        5: 近端校验失败;
17        6: 碰撞托盘;
18        7: 无法避障;
19    control_state: 叉车控制状态
20        0: ros控制;
21        1: 手动控制;
22    reserved: 预留位;
```

• 上位机主动请求

1. 心跳包

Plain Text

```
1 request: keep_connect
2 response:
3     hfls_version:HardwareVersion FirmwareVersion LoaderVersion SoftVersion
4 mark:
5     HardwareVersion 电源板版本号, FirmwareVersion 电源板固件版本号, LoaderVersion: loaderVersion, SoftVersion 导航系统版本号
6     保持5秒请求一次, 导航系统看门狗会检查串口连接状态
```

2. 整机关机

Plain Text

```
1 request: power_off
2 response: power_off:1
```

3. 整机重启

Plain Text

```
1 request: power_reboot
2 response: power_off:2
```

4. 重启导航

Plain Text

```
1 request: sys:reboot
2 response: initpose:0,x y radian
3 mark:
4     在开机/重启导航/切换地图时导航系统会主动上报(这几种情况导航会进行重定位;
5     x y radian: 机器坐标;
```

5. 待机

Plain Text

```
1 request: power_standby
2 response: power_off:5
3 mark:
4     低功耗模式时，下发控制关闭其他路电源；
```

6. 激光数据上报

Plain Text

```
1 request: switch_lidar[open?on:off]
2 response: laser[distance]
3 mark:
4     open: on : 打开; off : 关闭;
5     distance : 距离,单位 : m,精确到cm;
```

7. 导航主机连接wifi

因为安卓需和ros系统处于同一局域网才能调用http接口,所以在安卓连接wifi成功后需调用本接口使ros系统连接到同一网络

Plain Text

```
1 request: wifi[ssid name;pwd password]
2 response:
3     wifi:connect success 连接成功
4     wifi:connect fail 连接失败
5     wifi:connecting 连接进行中
6 mark:
7     name 是 WIFI 名称
8     password 是 WIFI 密码
9     仅支持 WPA/WPA2 Personal 协议的WIFI
```

8. 获取导航主机 Hostname

作为机器的唯一标识

Plain Text

```
1 request: hostname:get
2 response: sys:boot:robot-18
3 mark:
4     robot-18 是导航主机的 Hostname
```

9. 获取导航系统版本

Plain Text

```
1 request: sys:version
2 response: ver:3.0.0
```

10. 获取导航 wifi 名称和 ip

Plain Text

```
1 request: ip:request
2 response: ip:ssid:x.x.x.x/wlan:127.0.0.1
3 mark:
4     ssid: wifi名称
5     x.x.x.x: ip
6     wlan:127.0.0.1:网络未连接
7     RSN-3.1.7将wlan:127.0.0.1改为:
8     ip:notfound:127.0.0.1
```

11. 查询网口 ip

Plain Text

```
1 request: ip_lan:request
2 response: ip:enp2s0:192.168.10.2 / ip:lan:127.0.0.1
3 mark:
4     RSN-3.1.7改为:
5     lan:enp2s0:192.168.10.2 / lan:notfound:127.0.0.1
```

12.获取当前地图

Plain Text

```
1 request: nav:current_map
2 response: current_map[map alias]
3 mark:
4     map:当前地图md5
5     alias:别名 (如果没有别名response:current_map[map])
```

13.查询导航模式

Plain Text

```
1 request: model:request
2 response: model:x
3 mark:
4     x: 1(导航模式; 2(建图模式; model:3(增量建图模式;
5     开机后导航系统会主动上报一次;
6     切换模式时导航系统会自动上报;
```

14.切换导航模式

Plain Text

```
1 request: model:mode
2 response: model:x
3 mark:
4     | mode      | x      | remark          |
5     | ----- | ---- | ----- |
6     | navi       | 1      | 导航模式        |
7     | mapping    | 2      | 建图模式        |
8     | remap      | 3      | 增量建图模式    |
```

15.保存地图

Plain Text

```
1 request: save_map
2 response: model:1
3 mark:
4     建图完成后调用
```

16.切换地图

Plain Text

```
1 request: call_web[apply_map:map]
2 response: apply_map[map alias]
3         apply_map:failed
4 mark:
5     map : 地图md5
6     alias : 别名 (如果没有别名response: apply_map[map])
7     failed : 切换地图失败
8     地图切换完成后导航还会进行重定位,重定位完成上报`initpose:0,x y radian`,所有
    可以以此作为切换地图完成的标志;
```

17.获取已标定点的坐标

Plain Text

```
1 request: nav:get_flag_point[name]
2 response:
3     get_flag_point[x,y,radian,type,name]
4     get_flag_point:-1
5 mark:
6     x,y : 坐标
7     radian : 弧度
8     type : 类型
9     name : 点位名称
10    -1 : 找不到该标定点
```

18.设置标定点

Plain Text

```
1 request: nav:set_flag_point[x,y,radian,type,name]
2 response:
3     set_flag_point:result
4 mark:
5     x,y : 坐标;
6     radian : 弧度;
7     type : 充电桩类型固定:charge
8           配送点类型: delivery
9           出品点类型: production
10          回收点类型: recycle
11          路线点类型: normal
12          其他:可根据业务需求自定义类型
13     name : 点位名称
14     result:
15         0 : 成功
16         -1 : 名称长度大于50
17         -2 : 文件解析失败
18         -3 : 点位重复
```

19.删除标定点

Plain Text

```
1 request: nav:del_flag_point[name]
2 response:
3     del_flag_point:result
4 mark:
5     name : 要删除的点
6     result:
7         -1 : 找不到文件
8         -2 : 文件解析失败
9         -3 : 找不到点
```

20.重定位

(注: 发送名称重定位获取的是“标定位置”模式下的点位)

Plain Text

```
1 request:
2     nav:reloc[x,y,radian] (在(x,y,radian)坐标重定位;
3     nav:reloc_name[point] (在指定名称的点位重定位;
4 response: initpose:0,x y radian
5 mark:
6     在开机/重启导航/切换地图时导航系统会主动上报(这几种情况导航会进行重定位;
7     x y radian: 机器坐标;
```

21.绝对位置重定位

(注: 发送名称重定位获取的是“标定位置”模式下的点位)

Plain Text

```
1 request:
2     nav:reloc_absolute[point name]
3 response: initpose:0,x y radian
4 mark:
5     point name:点位名称;
6     不进行地图环境匹配,强制重定位到指定点;
```

22.绝对位置重定位(坐标重定位)

(注:只支持 RSNF-3.2.2 及以上版本)

Plain Text

```
1 request:
2     nav:reloc_abpoint[x,y,radian]
3 response: initpose:0,x y radian
4 mark:
5     point name:点位名称;
6     不进行地图环境匹配,强制重定位到指定坐标;
```

23.移动机器人

Plain Text

```
1 request:
2     move[distance,angle] [int, int] 类型
3     move[distance,angle,speed] [int, int, double] 类型
4 response: move:done:xx
5 mark:
6     1. distance 和 angle 必须要有一个为 0，也就是机器人只能走直线或者转弯，不能走弧线
7     2. 如果发 move[0,0] 表示停止移动
8     3. distance 单位为毫米
9     4. angle 单位为欧拉角，取值范围在 [-180,180] 之间
10    5. speed 为速度参数，不指定机器人会按照默认速度走（直线为 0.3 米每秒，转角为 40 度每秒）
11    秒)
12    6. speed 要大于 0，否则会按照默认速度处理
13    7. 如果指定 speed 且 distance 不为 0，speed 表示走直线的速度，单位是 米/秒
14    8. 如果指定 speed 且 angle 不为 0，speed 表示转角的速度，单位是 度/秒
15    9. 回复 move:done:16 : 直走完成
16    10. 回复 move:done:17 : 左转完成
17    11. 回复 move:done:18 : 右转完成
18    12. move:done:19: 后退完成
19    13. move:done:20: 取消
20    12. 有障碍物 : move_status:6
```

24.给定目标点名称导航

Plain Text

```
1 request: nav_point[前台]
2 response:
3     参考`ROS主动上报 - 2.导航状态`
4 mark:
5     发送目标点类型为`charge`时导航成功后对接充电桩
6     中文目标点需要在网页端“标定位置”选项下设置
7     中文字符编码方式utf8，非中文字符编码方式ASCII编码
```

25.坐标导航

Plain Text

```
1 request: goal:nav[x,y,radian]
2 response:
3     参考`ROS主动上报 - 2.导航状态`
4 mark:
5 x,y 为坐标 ,radian 是弧度
```

26.暂停导航

Plain Text

```
1 request: nav_pause
2 response:
3     参考`ROS主动上报 - 2.导航状态`
```

27.恢复导航

Plain Text

```
1 request: nav_resume
2 response:
3     参考`ROS主动上报 - 2.导航状态`
```

28.取消导航

Plain Text

```
1 request: nav_cancel
2 response:
3     参考`ROS主动上报 - 2.导航状态`
```

29.获取当前坐标

Plain Text

```
1 request : nav:get_pose
2 response:
3     nav:pose[x,y,radian] 当前机器人坐标
4     nav:pose:notfound 机器人正在定位中
5 mark:
6     x,y,radian 都是保留两位小数的浮点数，其中 x,y 是坐标， radian 是弧度
```

30.修改导航最大速度

Plain Text

```
1 request: max_vel[0.6]
2 mark:
3     范围:[0.3,1.0]
4     用于调整导航的最大速度
```

31.充电对接

Plain Text

```
1 request: dock:start
2 response:
3     参考`ROS主动上报 - 3.核心状态`
```

32.取消对接

Plain Text

```
1 request: dock:stop
2 response: 无
3 mark: 该指令在对接充电桩后退过程中有效
```

33.获取电池固定信息

Plain Text

```
1 request: get_battery_info
2 response:
3     battery_info{manufacturer nominal_voltage temperature cycle_times r
    ated_capacity full_capacity capacity health}
4 mark:
5     manufacturer : 制造商
6     nominal_voltage : 标称电压
7     temperature : 温度
8     cycle_times : 循环次数
9     rated_capacity : 额定容量
10    full_capacity : 满电容量
11    capacity : 当前容量
12    health : 健康程度
```

34. 获取电池动态信息

Plain Text

```
1 request: get_current_info[open?1:0]
2 response: current_info{55 202 0 0 1}
3         //电压 电流 适配器电流 警告 离线
4 mark:
5     open: 1 : 开; 0 : 关;
6     voltage : 电压;
7     current : 电流;
8     adapter_current : 适配器电流; (暂时未实现, 需要硬件支持)
9     warning : 警告;
10    offline : 0:离线 1:在线;
```

35. 获取导航动态参数

Plain Text

```
1 request: update_dynamic
2 response:
3     get_max_vel : speed
4     get_stop_time : second
5     get_global_p : p
6 mark:
7     speed : 当前导航速度
8     second : 遇到障碍物停留时间
9     p : 全局路径是否考虑临时障碍 1 : true 考虑 ; 0 :false
```

36.写入导航速度

Plain Text

```
1 request: write_max_vel[0.6]
2 response:
3     get_max_vel:0.6 :其中 0.6 为当前最大速度
4 mark:
5     范围:[0.3,1.0]
6     用于调整导航的最大速度
```

37.设置遇到障碍物停留时间

Plain Text

```
1 request: set_stop_time[5]
2 response:
3     get_stop_time:5.0
4 mark:
5     范围:[1,10]
```

38.设置全局路径是否考虑临时障碍

Plain Text

```
1 request: set_globalcost_p[1]
2 mark:
3     1 : 考虑临时障碍;
4     0 : 不考虑临时障碍;
```

39.自动上报坐标控制

Plain Text

```
1 request:nav:get_pose[open?on:off]
2 response:
3     nav:pose[x,y,radian] 当前机器人坐标
4     nav:pose:notfound 机器人正在定位中
5 mark:
6 open: on:打开;off:关闭
7     x,y,radian 都是保留两位小数的浮点数，其中 x,y 是坐标，radian 是弧度
8
```

40.设置其他机器人在本机地图上的位置和方向

Plain Text

```
1 request: robot_cost[x,y,z,line_speed,hostname,radius]
2 mark:
3     x,y : 坐标;
4     z : 弧度;
5     line_speed : 线速度;
6     hostname : 机器名;
7     radius : 膨胀半径(推荐在机器静止时设置0.03,机器移动时设置0.04);
```

41.设置CPU高性能模式

Plain Text

```
1 request: cpu_performance
```

42.设置就近停靠

Plain Text

```
1 request: set_tolerance[1/0]
2 mark:
3     1: 开启就近停靠;
4     0: 关闭就近停靠;
```

43.获取机器人当前所在特殊区

Plain Text

```
1 request: sendWeb[in_polygon]
2 response: in_polygon:special_area_name
3 mark:
4     在特殊区内时in_polygon:special_area_name;
5     不在特殊区内时in_polygon;;
```

44.取临停点

Plain Text

```
1 request: get_nearest
2 response: answer_nearest{nofind/x,y,radian}
3 mark:
4     nofind:取不到临停点;
5     x,y 为坐标 , radian 是弧度;
```

45.获取ROS主机开机时间

Plain Text

```
1 request: get_poweron_time
2 response: power_on_t:yyyy-MM-dd HH:mm:ss
```

46.过滤激光 & 3D 噪点范围

Plain Text

```
1 request: lidar_min[width]
2     mark:
3         width:宽度,浮点型,单位:米;
```

47.设置机器是否旋转(路径,目标点等被挡时)

Plain Text

```
1 request: avoid_obstacle[on/off]
2     mark:
3         on: 旋转;
4         off: 不旋转;
```

48.设置机器长宽

Plain Text

```
1 request: footprint[width,length]
2     mark:
3         width:宽度值,浮点型,单位:米;
4         length:长度值,浮点型,单位:米;
5         默认值: footprint[0,0];
```

49.设置机器半径

Plain Text

```
1 request: robot_radius[radius]
2     mark:
3         radius:半径值,浮点型,单位:米;
4         默认值: robot_radius[0]
```

50.设置机器从二维码离开时的方向和距离

Plain Text

```
1 request: agv_move[distance]
2     mark:
3         distance:
4             0: 机器直接离开;
5             正数: 机器向前移动指定距离后开始导航,浮点型,单位:米;
6             负数: 机器向后移动指定距离后开始导航,浮点型,单位:米;
7             默认值: 0.7;
```

51.设置机器二维码导航时的方向

Plain Text

```
1 request: agv_mode[direction]
2     mark:
3         0: 机器倒退着进行二维码导航;
4         1: 机器正向进行二维码导航;
5         默认值: 0;
```

52.固定路线导航

Plain Text

```
1 request: points_path[target_point_name]
2 response:
3     参考`ROS主动上报 - 2.导航状态`
4     mark:
5         target_point_name:目标点;
```

53.取消标签码导航

Plain Text

```
1 request: agv:stop
2 mark:取消标签码导航
```

54.查询固定路线路径

Plain Text

```
1 request: get_defined_plan[point]
2 response: getplan_dij:nofind
3         getplan_dij:path_point1 path_point2 path_point3...
4 mark:
5     point: 导航目标点
6     nofind: 无法生成路线
7     path_point: 路线点
```

55.固定路径点导航

Plain Text

```
1 request: list_point[1.12,2.13,3.11,4.13,5.11,2.01]
2         list_point_pre[1.12,2.13,3.11,4.13,5.11,2.01]
3 mark: 根据路径点组合成固定路线,按路线导航。上位机通过串口形式下发数据,由于数据长度
      最大为256,
4       一个点的坐标 (x,y,theta) 占用至少15位
5       每个数据保留了两位小数 (保留精度)
6       因此,一次最多传递14个坐标点。其中第一个点为当前坐标,最后一个点为目标点坐标。
7       保证路径不要经过穿过障碍,或者小角度急转弯,路径点间距可以在直线空间大一点,在复
      杂环境间
8       距小一点。
9       下发前,请计算该字符串的长度,请不要超过230
10      如果路线长度较长,超过14个坐标点,需分包发送,最后一包使用list_point指令,前面的
      路线用
11      list_point_pre指令
```

56.主动下发速度控制

Plain Text

```
1 request: app_vel[0.5,0.3]
2 mark: 其中第一位为线速度,第二位为角速度,均有正负值。正数代表前进或者左转,负数代表
      后退或者右转。
3       该接口下发频率请保证10hz左右
4       开始调试时,请从0.1开始,逐步增加到需要测试值,否则速度可能过大,影响安全。
```

57.设置固定路线绕障宽度

Plain Text

```
1
2 request: max_plan_dist[width]
3 mark: width: 单位:米
```

58.生成前往指定点的路线

JSON

```
1 request: get_plan_name [point]
2         get_plan_point [x, y, radian]
3 response: get_plan: error
4         get_plan: x1, y1, radian1, x2, y2, radian2...
5 mark: get_plan_name [point] 该接口只能传非固定路线模式的点位名称;
6      get_plan: error 生成路线失败, 一般为目标点或路径上有障碍物;
7      get_plan: x1, y1, x2, y2, ... get_plan: 后跟的是路线坐标
```

59. 设置深度相机是否参与动态避障 (RSNF1 3.2.3)

Plain Text

```
1 request: pc_switch [switch]
2 mark: switch 为0 则关闭深度相机避障, 为1 则打开深度相机避障
```

60. 设置是否根据路线弯曲调整最小速度 (RSNF1 3.2.4)

Plain Text

```
1 request: use_slow [switch]
2 mark: switch 为1 则使用动态最小速度, 为0 则不使用动态最小速度
```

61. 设置移动到目标点 (RSNF1 3.2.5)

Plain Text

```
1 request: move_goal [x, y, yaw]
2 response: move: done: 15
3 mark: 其中参数为目标点坐标, 请保留两位小数, 以0.1速度前进, 0.1速度旋转, 到达目标点
      通知
4      会考虑前方10cm障碍物, 则认为已经到达
```

62. 设置局部路径避障是否考虑地图层 (RSNF1 3.2.5)

Plain Text

```
1 request: local_static[on]
2 mark: 其中参数为 on 或者 off, on 代表使用地图, off 代表不使用地图
3       考虑地图层会更安全, 相对应的是, 由于墙体厚度变化, 通过宽度要求更大
```

洁霸独有

1. 开始清洁

Plain Text

```
1 request: clean_room[start:areaName]
2 mark:
3       areaName: 清洁区域名称
4 response:
5       clean_room[start:code,clean_id]
6 mark:
7       code:
8           -1: 失败, 没有找到房间名称;
9           -2: 失败, 无法生成任路线, 起点有障或者区域面积小于1平方;
10          -3: 失败, 没有找到房间, 至少三个点;
11          -4: 失败, 正在计算路线, 无法执行
```

叉车独有

1. 取卡板(调整横向位置偏差)

JSON

```
1 request: pallet:start[x,y,radians]
2 mark: x,y,radians为卡板点的坐标
```

2. 放卡板(调整横向位置偏差)

JSON

```
1 request: unload:start[x,y,radians]
2 mark: x,y,radians为卡板点的坐标
```

3. 进电梯(调整进梯前的横向位置偏差)

JSON

```
1 request: elevator_in:start[x,y,radians]
2 mark: x,y,radians为乘梯点的坐标
```

4. 出电梯

JSON

```
1 request:elevator_out:start[x,y,radians]
2 mark: x,y,radians为出梯点的坐标
```

5. 控制叉车臂

JSON

```
1 request: forklift_arm[1/0]
2 mark: 1:抬起叉车臂;0:放下叉车臂;
```

6. 退出卡板对接状态

JSON

```
1 request: pallet:stop
```

7. 设置避障距离

JSON

```
1 request: avoid:distance[0.3,0.3,0.2,0.2]
2 mark: 参数分别为前后左右;
```

8. 倒行

JSON

```
1 request: goal_back_point[x,y,theta]
2 response: 同正常导航
3 mark: 倒行时叉车不会调整角度,需先将车尾朝向目标点方向;
```

9. 设置叉车臂处激光角度

JSON

```
1 request: set_arm_laser[angle]
2 mark: angle范围:[20,160]
```

10. 检测目标位置是否有障碍物

JSON

```
1 request: check:area_reachable[positionCenter[0],positionCenter[1],positionCenter[2],distanceToFront,distanceToBack,distanceToLeft,distanceToRight]
2 response: area_reachable:x
3 mark: positionCenter为目标点坐标;
4         distanceToXX为需判断目标点前后左右一定范围内是否有障碍物的距离;
5         area_reachable:x: x为1时表示无障碍物,0表示有障碍物;
```

叉车取卡板流程

JSON

1 到达指定位置后取卡板	<code>android send to ros -> pallet:start[x,y,radians]</code>
2 开始调整横向偏差	<code>forklift{1 0 1 0 0} <- ros send to android</code>
3 横向偏差调整完成	<code>forklift{1 0 3 0 0} <- ros send to android</code>
4 安卓发送放下叉车臂	<code>android send to ros -> forklift_arm[0]</code>
5 放下叉车臂后开始对接卡板	<code>forklift{0 0 3 0 0} <- ros send to android</code>
6 对接卡板完成	<code>forklift{2 0 2 0 0} <- ros send to android</code>
7 抬起叉车臂	<code>android send to ros -> forklift_arm[1]</code>
8 已抬起叉车臂	<code>forklift{1 0 0 0 0} <- ros send to android</code>
9 离开卡板位置	<code>android send to ros -> move[1400,0]</code>
10 移动完成	<code>move:done:16 <- ros send to android</code>
11 <code>mark:move[1400,0]</code> 指前进1.4m, 此处仅供参考, 实际应计算卡板点到卡板对接点的距离;	

叉车放卡板流程

JSON

1 到达指定位置后放卡板	<code>android send to ros -> unload:start[x,y,radians]</code>
2 开始调整横向偏差	<code>forklift{1 0 1 0 0} <- ros send to android</code>
3 横向偏差调整完成	<code>forklift{1 0 3 0 0} <- ros send to android</code>
4 安卓发送抬起叉车臂	<code>android send to ros -> forklift_arm[1]</code>
5 开始调整到卡板点的纵向偏差	<code>forklift{1 0 1 0 0} <- ros send to android</code>
6 到达卡板点位置	<code>forklift{1 0 2 0 0} <- ros send to android</code>
7 放下叉车臂	<code>android send to ros -> forklift_arm[0]</code>
8 已放下叉车臂	<code>forklift{2 0 0 0 0} <- ros send to android</code>
9 离开卡板位置	<code>android send to ros -> move[1400,0]</code>
10 移动完成	<code>move:done:16 <- ros send to android</code>
11 <code>mark:move[1400,0]</code> 指前进1.4m, 此处仅供参考, 实际应计算卡板点到卡板对接点的距离;	

透传指令

Plain Text

```
1 request: send_to_base[data]
2 response: base_data:{}
3 mark: 导航转发,发送给电源板
```

● 电源板升级

1. 上传升级文件

Plain Text

```
1 web端将升级文件上传;
```

2. 进入 loading 模式

Plain Text

```
1 request: base_upgrade[start]
2 response: base_upgrade:loading
```

3. 升级

Plain Text

```
1 request: base_upgrade[affirm]
2 response: base_upgrade:success
```

测试里程碑

Plain Text

```
1 roslaunch yoyo_node test_get_odom.launch  
2 rostopic echo /mobile_base/debug/odom_pose2d
```

测试陀螺仪

Plain Text

```
1 roslaunch yoyo_node test_get_yaw.launch  
2 rostopic echo /mobile_base/debug/imu_angle
```